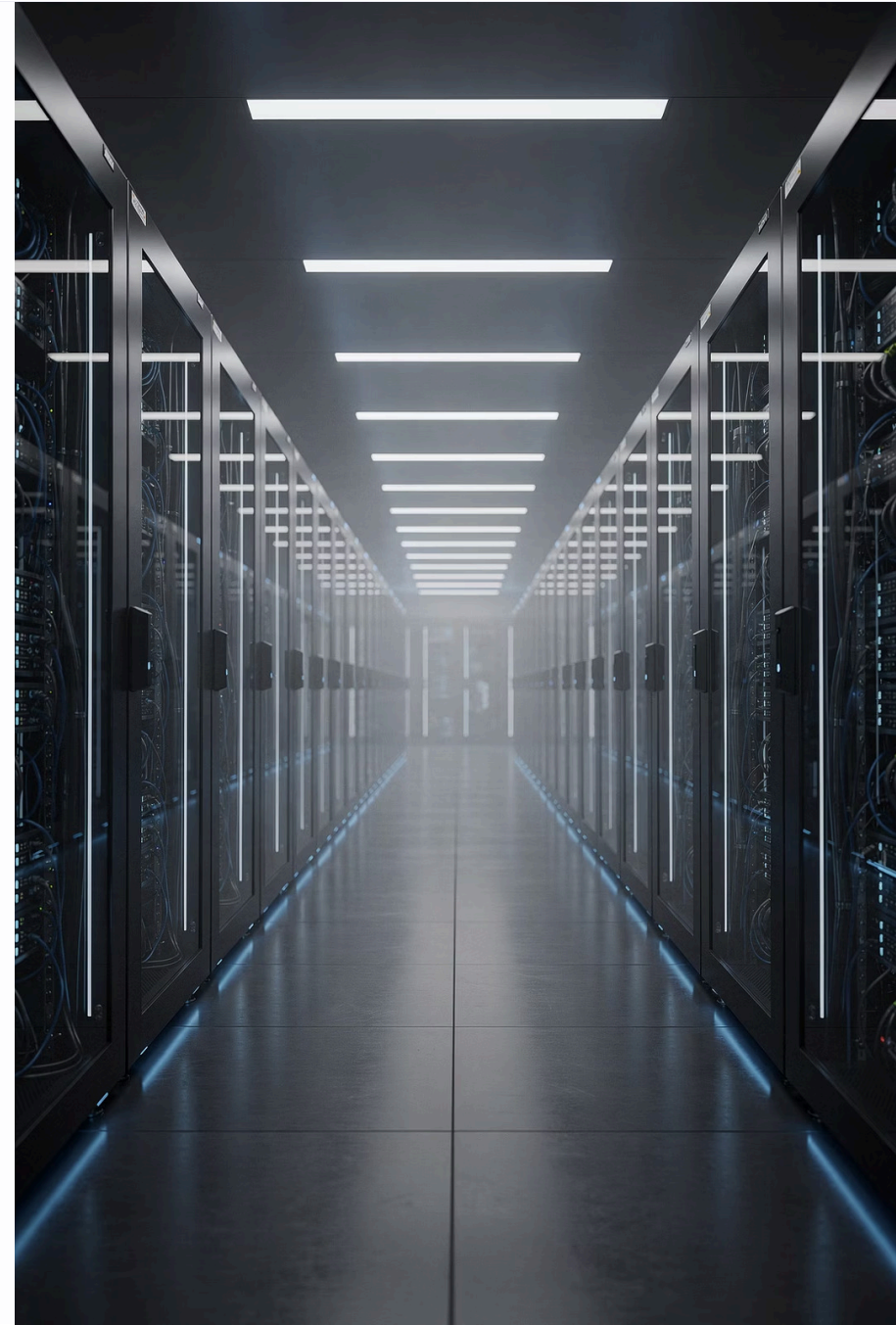


Infrastructure Platform Engineering

Day 4 · Modules 6 & 7 — From siloed infrastructure to a living, breathing platform that powers your entire engineering organization.

PLATFORM ENGINEERING

MODULE 6 & 7



What We'll Cover Today

Two modules, one story: how to stop fighting your infrastructure and start building a platform your teams actually love to use.



Module 6: Infrastructure Platform Engineering

Moving from silos to APIs, cross-cutting concerns, everything-as-code, and real-world examples.



Module 7: Building a Minimum Viable Platform

Why MVP matters, what it looks like, and a practical guide to building one that scales.



Why Infrastructure Is the Foundation of Everything

Every feature your team ships, every customer interaction, every business metric ultimately rests on infrastructure. Yet it is often the most under-invested layer in the stack.

The Hidden Risk

When infrastructure is neglected, it becomes a source of outages, slow deployments, and developer frustration — not a competitive advantage.

The Opportunity

Well-engineered infrastructure acts as a multiplier — every team moves faster, safer, and more confidently when the platform underneath them is solid.

The Real Cost of Neglecting Infrastructure

Think about the last time a deployment was delayed because of a manual provisioning step, or a weekend was lost to an outage caused by config drift. These are symptoms of infrastructure debt.



Slow Delivery

Teams wait days for environments to be provisioned manually. Developer velocity drops as infrastructure becomes a bottleneck.



Fragile Systems

Inconsistent configs across environments cause "works on my machine" failures. Production surprises become routine.



Developer Burnout

Engineers spend time fighting infrastructure instead of building product. Frustration compounds over time and talent walks out the door.

MODULE 6

From Silos to APIs

The shift that changes everything about how infrastructure is consumed.

The Old World: Infrastructure Silos

How It Used to Work

Each team owned their own servers, their own scripts, their own way of doing things. Networking team over here. Storage team over there. Cloud team somewhere else.

Result: duplicated effort, inconsistent standards, and weeks of coordination just to spin up a new service.

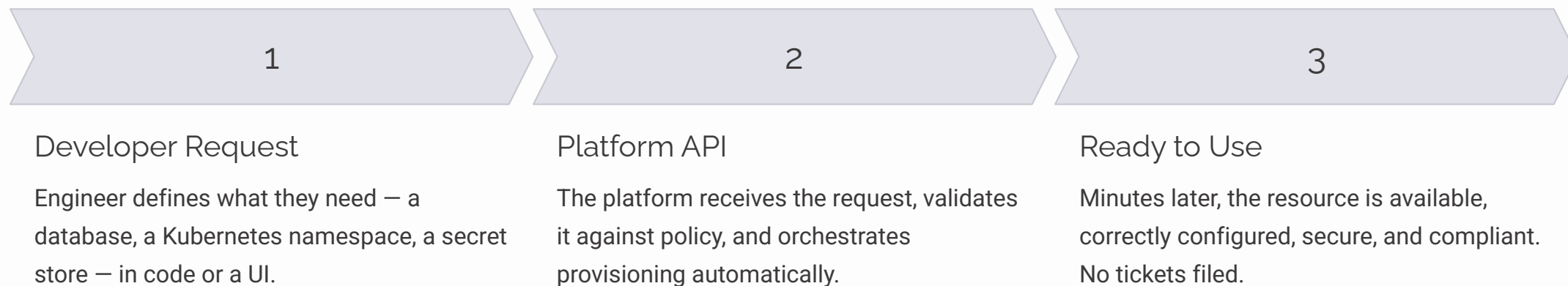
A Real-World Example

Imagine a product team at a bank wanting to launch a new microservice. They file a ticket to Networking, another to Storage, another to the DBA team. Three weeks later, they still don't have an environment.

Meanwhile, a competitor has already shipped. This is the cost of silos.

The New World: Infrastructure as APIs

Platform engineering flips the model. Instead of filing tickets, developers call APIs. Infrastructure becomes a **self-service product**, not a gated resource.



This is how teams like Spotify, Netflix, and Airbnb have scaled infrastructure without scaling their ops headcount linearly.

Cross-Cutting Concerns: The Things Everyone Needs

Some infrastructure capabilities are not unique to one team — they are needed everywhere. Platform engineering centralizes these so every team gets them right, automatically.



Security

TLS termination, secret management, and IAM policies baked into every provisioned resource — not bolted on later.



Observability

Logging, metrics, and tracing configured by default. Every service is observable from day one without extra setup.



Networking

Service mesh, DNS, and load balancing handled at the platform layer. Teams don't think about network plumbing.



Compliance

Policy guardrails enforced automatically — tagging, cost allocation, and regulatory controls are non-negotiable defaults.



Everything-as-Code

If it isn't in version control, it doesn't exist. Everything-as-code is the principle that all infrastructure — networks, policies, pipelines, monitoring — is defined, versioned, and deployed using code.

Why Everything-as-Code Matters

The Problem with Manual Changes

Someone SSH'd into a production server six months ago and made a "quick fix." Nobody wrote it down. Now it's causing an outage and nobody knows why the environment is different from staging.

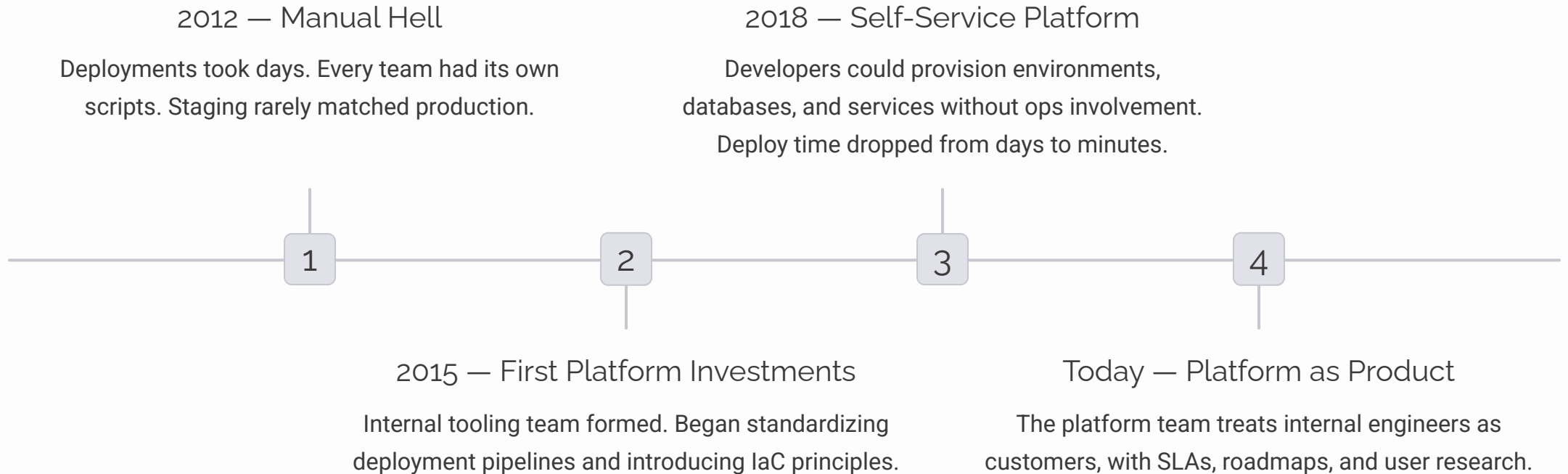
Manual changes are undocumented, unrepeatable, and invisible — the three most dangerous qualities in infrastructure.

Code Solves All Three

- **Documented** — every change has a commit message and a PR review.
- **Repeatable** — the same Terraform module produces identical environments every time.
- **Visible** — git history is your audit log. Rollback is a revert away.

Real-World Example: How Airbnb Tamed Infrastructure Chaos

Airbnb grew from a handful of engineers to thousands — and their infrastructure nearly broke them. Here is how platform engineering saved the day.



The Limits of Developer Portals

Developer portals like Backstage are popular — and genuinely useful. But there is a common trap: teams build a beautiful UI on top of broken, manual processes and call it a platform.

⚠ A developer portal is a **window into your platform**, not the platform itself. If the underlying infrastructure is manual, siloed, or inconsistent, the portal just makes the chaos prettier.

What Portals Do Well

Surface service catalog information, documentation, and status.
Improve discoverability and reduce context switching.

What Portals Cannot Fix

Broken CI/CD pipelines, manual provisioning, missing observability, and siloed teams. These are platform problems, not UI problems.

Module 6 Key Takeaways



Infrastructure Is Strategic

It is not a cost center — it is the foundation of your engineering capability and competitive speed.



Neglect Has Real Costs

Slow delivery, fragile systems, and burned-out engineers are predictable outcomes of under-investing in platform work.



Portals Are Not Platforms

Build the right abstractions and APIs first. A portal is useful only when the platform behind it is solid and self-service.

MODULE 7

Building a Minimum Viable Platform

Start small, deliver value fast, and grow the platform with your organization.

Why "MVP" Is the Right Mental Model

Most platform teams fail not because they lack ambition, but because they try to build everything at once. They spend 18 months building the perfect platform — and ship nothing.

The Big Bang Trap

Teams design a grand, enterprise-wide platform. Years pass, requirements change, engineers leave. By the time it ships, the business has moved on and adoption is low.

The MVP Mindset

Ship something useful in weeks, not years. Get real feedback from real developers. Iterate based on what actually causes pain — not what you assumed would.

This is how Spotify's Backstage, Netflix's Spinnaker, and Uber's internal platforms were all born: solving one problem well, then expanding.

What Is a Platform MVP?

A Minimum Viable Platform is the smallest set of capabilities that delivers meaningful value to your internal developers — reducing toil, improving reliability, or accelerating delivery.

1

One Golden Path

A single, opinionated, well-supported way to deploy a service. Not ten ways — one great way that covers the most common use case.

2

Self-Service Core

Developers can provision the essentials — compute, storage, secrets — without filing a ticket or waiting for an ops engineer.

3

Observability by Default

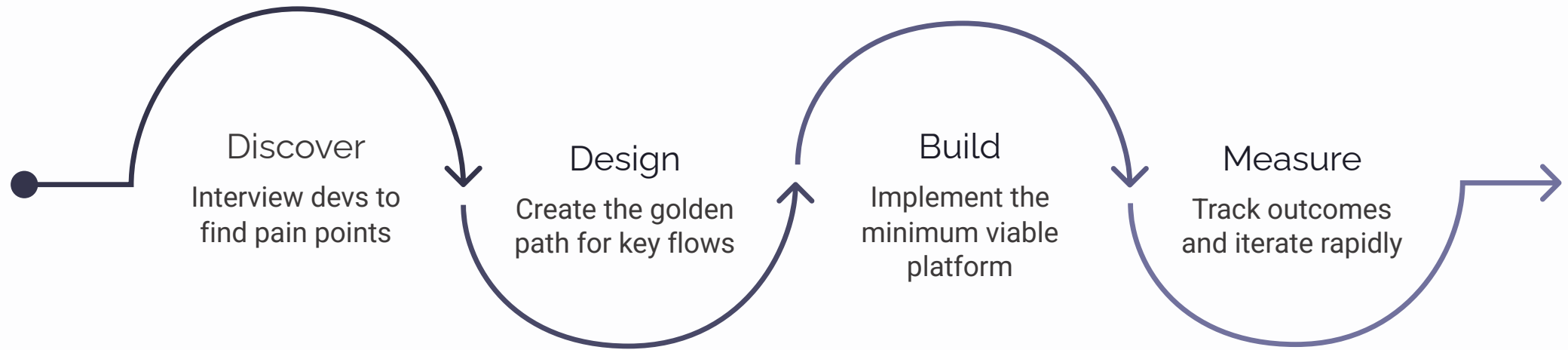
Every service deployed via the platform gets logging and basic metrics automatically. No extra configuration required.

4

Feedback Loop

A lightweight way to collect developer feedback — a Slack channel, a survey, or regular office hours with the platform team.

How to Build It: A Practical Approach



The most important step is the first one: talk to your developers. Understand where they lose the most time before writing a single line of platform code. Then build the minimum thing that eliminates that specific pain — and ship it.

Real-World Example: A Mid-Size Fintech's MVP Journey

A fintech company with 200 engineers was struggling with inconsistent deployments and a six-hour average time to provision a new service. Here is how they built their MVP in 90 days.



Week 1–2: Discovery

Platform team ran interviews with 20 engineers. Top complaint: provisioning a new microservice required coordination across 4 teams and took up to a week.



Week 3–6: Golden Path Design

Designed a single Terraform module and CI/CD template for a standard backend service. Kept it opinionated — one language runtime, one container platform, one logging setup.



Week 7–12: Ship and Iterate

Onboarded 3 pilot teams. Provisioning time dropped from 5 days to 45 minutes. Collected feedback weekly and shipped improvements in two-week cycles.

Designing for Scale from Day One

An MVP doesn't mean throwaway. Build your platform so it can grow — without requiring a complete rewrite when you add the next capability.

Abstractions Over Implementation

Expose logical concepts (a "database", a "cache") not implementation details (RDS version, Redis config). This lets you swap the underlying technology without breaking consumers.

Versioned APIs

Treat your internal platform like a public product. Version your APIs so teams can upgrade on their own schedule without being forced into breaking changes.

Modular Components

Build capabilities as independent modules — CI/CD, secrets, networking — so teams can adopt them incrementally rather than all-or-nothing.



Key Takeaways & Next Steps



Infrastructure is strategic, not optional

Neglecting it compounds debt — in speed, reliability, and talent retention. Invest early and treat it as a product.



Move from silos and tickets to APIs and self-service

The shift to platform engineering is a cultural and technical change. Start with the highest-friction workflows first.



Start with an MVP — ship value in weeks, not years

Pick one golden path, eliminate one major pain point, and build trust with your internal customers before expanding scope.



Design for iteration and scale from the beginning

Versioned APIs, modular components, and strong abstractions let your MVP grow into a mature platform without a rewrite.